Machine Data Learning (MDL)

Complete Exam Study Guide

Based on All Lectures: Jan 3 – Feb 21, 2026

Modules Covered:

Module 1: Introduction to AI & Machine Learning
Module 2: Propositional Logic & Reasoning
Module 3: ML Fundamentals: Generalization, Bias-Variance
Module 4: Data Preprocessing & Feature Engineering
Module 5: Cross-Validation & Regularization
Module 6: Feature Selection & Dimensionality Reduction
Module 7: Data Mining & Association Rules
Module 8: Classification & Naive Bayes

Reference Books:

Artificial Intelligence: A Modern Approach – Stuart Russell & Peter Norvig
Python Machine Learning by Example – freely available

Spring 2026 – IIIT

Contents

1 Introduction to Artificial Intelligence & Machine Learning 3
1.1 What is Intelligence? . . . 3
1.2 Two Dimensions of AI . . . 4
1.3 Simple & Complex Machines . . . 4
1.4 Applications of AI . . . 4
1.5 Ethics and Fairness in AI . . . 5
1.6 What is Machine Learning? . . . 5
1.7 Types of Machine Learning . . . 5
1.8 Neural Networks & Deep Learning . . . 6
1.9 AlphaGo & Reinforcement Learning . . . 7

2 Propositional Logic & Reasoning 8
2.1 Why Formal Languages? . . . 8
2.2 Hierarchy of Logic . . . 8
2.3 Propositions . . . 9
2.4 Logical Operators (Connectives) . . . 9
2.5 Truth Tables . . . 9
2.6 Logical Equivalences . . . 10
2.7 Validity & Satisfiability . . . 10
2.8 Logical Deduction & Arguments . . . 10
2.9 Inference Rules . . . 11
2.10 Proof in Propositional Logic . . . 11
2.11 Soundness & Completeness . . . 11
2.12 Normal Forms . . . 12

3 Machine Learning Fundamentals 13
3.1 Generalization . . . 13
3.2 Overfitting & Underfitting . . . 13
3.3 Bias-Variance Tradeoff . . . 14
3.4 Training Data vs Test Data . . . 14
3.5 Mean Squared Error (MSE) . . . 14
3.6 Goodness of Fit . . . 15

4 Data Preprocessing & Feature Engineering 16
4.1 Step 1: Data Profiling . . . 16
4.2 Step 2: Data Cleansing . . . 16
4.3 Step 3: Data Transformation . . . 17
4.3.1 Feature Scaling . . . 17
4.3.2 Feature Encoding . . . 17
4.4 Step 4: Data Reduction . . . 18
4.5 Step 5: Data Enrichment . . . 18
4.6 Data Leakage . . . 18

5 Cross-Validation & Regularization 19
5.1 Cross-Validation . . . 19
5.1.1 K-Fold Cross-Validation . . . 19
5.1.2 Leave-One-Out Cross-Validation (LOOCV) . . . 19
5.1.3 Nested Cross-Validation . . . 20
5.2 Regularization . . . 20

5.3 Other Techniques to Prevent Overfitting 21

6 Feature Selection & Dimensionality Reduction **22**
6.1 Why Feature Selection? 22
6.2 Feature Selection Methods 22
6.3 Dimensionality Reduction 22
6.3.1 Principal Component Analysis (PCA) 23
6.4 CRISP-DM Framework 23

7 Data Mining & Association Rules **24**
7.1 Data Science vs Data Analytics vs Data Mining 24
7.2 Association Rules 24
7.3 Measures for Association Rules 24
7.4 Frequent Itemsets & The Apriori Algorithm 25

8 Classification & Naive Bayes Classifier **27**
8.1 What is Classification? 27
8.2 Evaluating a Classifier 27
8.3 Bayes Classifier 28
8.4 Naive Bayes Classifier 28

9 Quick Reference Summary **30**
9.1 ML Types at a Glance 30
9.2 Key Formulas Cheat Sheet 30
9.3 Logic Quick Reference 31
9.4 Data Preprocessing Pipeline 31

10 Important Potential Exam Questions **32**

1 Introduction to Artificial Intelligence & Machine Learning

What is Artificial Intelligence?

Artificial Intelligence (AI) is the science and engineering of making machines (computers, robots, and other systems) that can think and act like humans. More formally:

> *"AI is the science and engineering of making machines intelligent, enabling them to take actions and decisions that humans typically do."* – Stanford Definition

AI is considered **one of the most profound undertakings in science**, affecting every aspect of human life. In 2024, Nobel Prizes were awarded to AI/ML researchers, confirming the field's importance.

1.1 What is Intelligence?

Definition: Intelligence

Intelligence is the capacity for:

- Abstraction
- Logic & Reasoning
- Understanding
- Self-awareness
- Learning
- Emotional knowledge
- Planning
- Creativity
- Critical thinking
- Problem solving

Key Insight: We want to make machines capable of these things — that is the goal of AI.

How Humans Learn

Humans learn by:

1. Being exposed to the surrounding environment
2. Learning from observations about the environment
3. Learning to act and make decisions

Goal of AI: Can machines think? Can machines make decisions? Can machines learn from observations like humans do?

1.2 Two Dimensions of AI

	Human-like	Rational (Optimal)
Thinking	Cognitive modeling – model how humans think	Logical reasoning – think correctly using logic
Acting	Turing Test – behave indistinguishably from humans	Rational agents – always do the "right thing"

The Turing Test

To pass the Turing Test, a computer needs:

- **Natural Language Processing (NLP)** – to communicate in human language
- **Knowledge Representation** – to store and organize knowledge
- **Logical/Automated Reasoning** – to draw inferences and conclusions
- **Machine Learning** – to learn from observations/data
- **Computer Vision & Robotics** – for the "Total Turing Test" (physical interaction)

1.3 Simple & Complex Machines

Machines

A **machine** is a physical system that uses power to apply forces and control movement to perform an action, making life easier.
Six types of simple machines: Lever, Wheel & Axle, Pulley, Inclined Plane, Wedge, Screw.
Complex machines: Combinations of simple machines – sewing machines, cars, trains, computers, robots.

1.4 Applications of AI

AI Applications in the Real World

- Autonomous driving
- Large Language Models (ChatGPT)
- Computer Vision
- Crowdsourcing & Crowdfunding
- Online advertisement
- Healthcare & Medical diagnosis
- Finance & Stock prediction
- Recommendation systems (Netflix, Amazon)
- Judicial systems & Recruitment
- Agriculture & Smart cities
- Wildlife safety & Poaching prevention
- Supply chain management

1.5 Ethics and Fairness in AI

Important: AI Ethics

Key Questions:

- Will AI take fair decisions? Many instances show biases exist.
- Who takes responsibility for AI failures – the coder, the company, or the user?
- **Uber Accident (2018):** A self-driving car hit and killed a pedestrian. Debate about whether it was a sensor failure, software failure, or motor failure. Most likely the AI estimated the woman was riding a bicycle and would pass in time, but she was walking with the cycle.

Takeaway: AI models are NOT fair by default. They must be intentionally *designed* to be fair. Responsible AI development (transparent, explainable, fair) is essential.

1.6 What is Machine Learning?

Definition: Machine Learning (ML)

Machine Learning is a scientific study of algorithms and statistical models that computer systems use to perform specific tasks effectively **without using explicit instructions**, relying on patterns and inferences from data instead.
Philosophy: Everything in the world is governed by mathematics. ML algorithms learn and replicate these mathematical patterns from data.
Key Idea: Humans learn from observations → Machines learn from **data**.

1.7 Types of Machine Learning

Type	Description	Examples
Supervised Learning	Training on **labeled data**; learn mapping from inputs to known outputs	Classification, Regression, Image recognition
Unsupervised Learning	Training on **unlabeled data**; discover hidden patterns/structure	Clustering, Anomaly detection, Dimensionality reduction
Reinforcement Learning	Agent learns by **interacting with environment**; receives rewards/penalties	Game playing (AlphaGo), Robotics, Autonomous vehicles

Example: Cricket Ball vs Tennis Ball

Suppose we want to classify whether a ball is a **cricket ball** or a **tennis ball**.
Features (data): diameter and weight of various balls.
Observations:

- Cricket ball: diameter $\approx$ 2.8–2.9 inches, weight $\approx$ 155–163 grams (heavier)
- Tennis ball: diameter $\approx$ 2.57–2.70 inches, weight $\approx$ 56–59 grams (lighter)

If given a new ball with diameter = 2.75 inches and weight = 120 grams → likely a **cricket ball** (since tennis balls are generally below 60 grams).
This is **supervised learning** because we already know the labels (cricket/tennis) of training data.

1.8 Neural Networks & Deep Learning

Neural Networks

Perceptron (simplest neural unit): Mimics a single neuron.

1. Input features: $x_1, x_2, \ldots, x_n$
2. Weights: $w_1, w_2, \ldots, w_n$ and bias w_0
3. Output: $y = f(w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n)$

Where f is an **activation function** that determines the output class.
Neural Network: Multiple interconnected perceptrons (neurons) arranged in layers.
Deep Neural Network: Neural network with **many hidden layers** — this is what makes "Deep Learning" powerful.

Perceptron Formula

For two features x_1 (diameter) and x_2 (weight):

$$y = \begin{cases} 1 \text{ (cricket ball)} & \text{if } w_0 + w_1x_1 + w_2x_2 \geq \text{threshold} \\ 0 \text{ (tennis ball)} & \text{otherwise} \end{cases}$$

Bias (w_0): A constant added to allow the decision boundary to shift. Without it, the boundary must pass through the origin.
Why we need algorithms: In complex real-world data, we cannot manually set weights and biases. We need **learning algorithms** to determine the optimal $w_0, w_1, w_2, \ldots$

Scale of Modern Neural Networks

Model/Year	Parameters
Simple perceptron (our example)	3 (w_0, w_1, w_2)
Early networks (2015-16)	$<$ 1 billion
GPT-2 (2017)	94 million
GPT-4 (March 2023)	1.76 **trillion**
Llama (May 2023)	175 billion
Human brain neural connections	$\sim 100\times$ larger

1.9 AlphaGo & Reinforcement Learning

AlphaGo – A Landmark in AI

AlphaGo was developed by **DeepMind** (Google). It defeated the human world champion in **Go** (a complex strategy board game with 10^{170} possible positions).
How it works:

1. **Monte Carlo Tree Search (MCTS):** A randomized algorithm that explores the game tree by simulation rather than exhaustive search.
2. **Deep Neural Networks:** Used to evaluate board positions and select moves.
3. **Self-play (Reinforcement Learning):** AlphaGo played millions of games against itself to improve. Winner gets reward, loser gets penalty.

AlphaZero: Later version that learned *without any human data* — purely through self-play.
AlphaFold: Used similar ML techniques to solve the **protein folding problem** (predicting 3D protein structure from amino acid sequence), winning the Nobel Chemistry Prize in 2024.

Levinthal's Paradox

Proteins can fold into their native conformation very quickly, despite the astronomically large number of possible conformations. This paradox was a long-standing challenge in biology until **AlphaFold** used ML to predict protein structures accurately.

2 Propositional Logic & Reasoning

Why Logic Matters in AI

AI was initially built on **formal logic**. Before ML became dominant, scientists developed logical systems to make computers reason.
Knowledge-Based AI requires:

- **Knowledge Representation:** Expressing knowledge explicitly in a computer-tractable way
- **Logical Reasoning:** Drawing inferences from that knowledge
- **Automated Reasoning:** Having computers prove/disprove statements systematically

2.1 Why Formal Languages?

Problems with Natural Language

Natural languages have three critical issues:

1. **Ambiguity:** "He saw a man with a telescope" — Who has the telescope?
2. **Implicit assumptions:** We assume things while speaking that the other person may not know.
3. **Multiple interpretations:** Sentences can have 2–3 different meanings.

Solution: Use **formal languages** (like propositional logic) that are:

- Rigorous and precise
- Reduce human errors
- Make all assumptions explicit
- Allow confident, verifiable derivations

2.2 Hierarchy of Logic

Order	Name	Description
Zeroth-order	Propositional Logic	Deals with propositions (statements that are T or F) and their combinations via logical operators
First-order	Predicate Logic	Adds quantifiers ($\forall$, $\exists$) over **individual elements**
Second-order	Second-order Logic	Quantifies over **sets** of elements
Higher-order	Higher-order Logics	Quantifies over sets of sets, etc.

Course Focus

In this course, we focus on **Propositional Logic (Zeroth-order Logic)**. All other logics are built on top of propositional logic.

2.3 Propositions

Definition: Proposition

A **proposition** is a declarative statement that is either **True (T)** or **False (F)**, but not both.
Properties:

- Simple, indivisible statements
- Form a representation for basic facts
- Denoted by lowercase letters: $p, q, r, s, \ldots$

Which are Propositions?

Statement	Proposition?	Reason
"All humans have two eyes"	Yes	Declarative, has truth value (True)
"All humans have four eyes"	Yes	Declarative, has truth value (False)
"7301 is a prime number"	Yes	Can be verified as True or False
"Good morning!"	No	Exclamation, not declarative
"What time is it?"	No	Question, not declarative
"$x + 3 = 7$"	No	Contains variable; truth depends on x

2.4 Logical Operators (Connectives)

Operator	Symbol	Meaning	Read as
Negation	$\neg p$	NOT p; flips truth value	"not p"
Conjunction	$p \wedge q$	p AND q; true only when both are true	"p and q"
Disjunction	$p \vee q$	p OR q; true when at least one is true	"p or q"
Implication	$p \Rightarrow q$	If p then q	"p implies q"
Biconditional	$p \Leftrightarrow q$	p if and only if q	"p iff q"

2.5 Truth Tables

Complete Truth Table for All Operators

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \Rightarrow q$	$p \Leftrightarrow q$
T	T	F	T	T	T	T
T	F	F	F	T	**F**	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Critical Point about Implication ($p \Rightarrow q$):

- Only **False** when p is True and q is False.
- When p is False, the implication is **always True** (vacuous truth).
- p is the **sufficient condition** for q.
- q is the **necessary condition** for p.

2.6 Logical Equivalences

Key Equivalences to Memorize

$$p \Rightarrow q \equiv \neg p \vee q \tag{1}$$
$$p \Leftrightarrow q \equiv (p \Rightarrow q) \wedge (q \Rightarrow p) \tag{2}$$
$$\neg(p \wedge q) \equiv \neg p \vee \neg q \quad \text{(De Morgan's Law)} \tag{3}$$
$$\neg(p \vee q) \equiv \neg p \wedge \neg q \quad \text{(De Morgan's Law)} \tag{4}$$
$$p \Rightarrow q \equiv \neg q \Rightarrow \neg p \quad \text{(Contrapositive)} \tag{5}$$

2.7 Validity & Satisfiability

Validity, Satisfiability, Unsatisfiability

- **Valid (Tautology):** A sentence that is **True under ALL possible assignments** of truth values to its variables.
  Example: $p \vee \neg p$ is always True.
- **Satisfiable:** A sentence for which **there exists at least one assignment** that makes it True.
  Example: $p \wedge q$ is satisfiable (True when $p = T, q = T$).
- **Unsatisfiable (Contradiction):** A sentence that is **False under ALL possible assignments**.
  Example: $p \wedge \neg p$ is always False.

Relationship

- Every valid sentence is satisfiable, but not every satisfiable sentence is valid.
- A sentence α is valid $\Leftrightarrow \neg\alpha$ is unsatisfiable.
- $\alpha \models \beta$ ("α entails β") means: whenever α is True, β must also be True.

2.8 Logical Deduction & Arguments

Valid Arguments vs. True Conclusions

An argument consists of **premises** (hypotheses) and a **conclusion**.
Important Distinction:

- A **valid argument** means the reasoning/deduction steps are correct.
- But if the **premises are false**, you can still get a **false conclusion** from valid reasoning!

Example:

1. Premise: "All humans have four eyes" (FALSE premise)
2. Premise: "Sita is a human" (TRUE)
3. Conclusion: "Sita has four eyes" (FALSE conclusion, but reasoning is VALID)

Takeaway: For correct deductions, **ALL hypotheses must be correct**, then conclusions will be correct.

2.9 Inference Rules

Key Inference Rules

1. **Modus Ponens:** From p and $p \Rightarrow q$, conclude q.
$$\frac{p, \quad p \Rightarrow q}{q}$$
2. **Modus Tollens:** From $\neg q$ and $p \Rightarrow q$, conclude $\neg p$.
$$\frac{\neg q, \quad p \Rightarrow q}{\neg p}$$
3. **And-Elimination:** From $p \wedge q$, conclude p (or q).
$$\frac{p \wedge q}{p}$$
4. **And-Introduction:** From p and q, conclude $p \wedge q$.
$$\frac{p, \quad q}{p \wedge q}$$
5. **Or-Introduction:** From p, conclude $p \vee q$ (for any q).
$$\frac{p}{p \vee q}$$
6. **Resolution:** From $p \vee q$ and $\neg p \vee r$, conclude $q \vee r$.
$$\frac{p \vee q, \quad \neg p \vee r}{q \vee r}$$

2.10 Proof in Propositional Logic

Formal Proof

A **formal proof** of $\alpha \Rightarrow \beta$ is a sequence of steps where each step is:

1. An **axiom** or **premise** of α, OR
2. Derived from **previous steps** using **rules of inference**

If you reach β following these rules, then β is provable from α, written as $\alpha \vdash \beta$.

2.11 Soundness & Completeness

Soundness & Completeness of Logical Systems

Property	Meaning
Sound	If you can prove β from α, then β is actually true whenever α is true. (You never prove something false)
Semantically Complete	If something is true, you can prove it. (All true things are provable)
Syntactically Complete	For **any** statement S, you can prove either S is true OR $\neg S$ is true. (You can decide the truth of any statement — much stronger!)
Decidable	There exists a **finite procedure** to determine truth/falsity.

Key Results:

- **Zeroth-order logic (Propositional):** Semantically complete, syntactically complete, AND decidable (truth tables give 2^n rows, but it's finite).
- **First-order logic:** Semantically complete (Gödel's completeness theorem) but **NOT** syntactically complete and **NOT** decidable.

2.12 Normal Forms

CNF and DNF

Any propositional sentence can be converted to standard forms:
Conjunctive Normal Form (CNF):

- A **conjunction** (AND) of **clauses**
- Each clause is a **disjunction** (OR) of literals
- Example: $(p \vee \neg q) \wedge (\neg p \vee r) \wedge (q \vee r)$

Disjunctive Normal Form (DNF):

- A **disjunction** (OR) of **terms**
- Each term is a **conjunction** (AND) of literals
- Example: $(p \wedge q) \vee (\neg p \wedge r) \vee (q \wedge \neg r)$

Key conversion: $p \Rightarrow q \equiv \neg p \vee q$ (using Equation 1)
Advantage: Converting to CNF/DNF gives a **standard structure** that is easier to work with in automated reasoning.

3 Machine Learning Fundamentals

3.1 Generalization

Definition: Generalization

Generalization is the ability of a machine learning model to perform well on **new, unseen data** — not just the data it was trained on.
Analogy: Like a student who truly *understands* concepts vs. one who merely *memorizes* specific exam questions. The first student generalizes; the second does not.

Cat Recognition Example

If you train a model on photos of cats:

- A **good model** recognizes a cat even in a new photo with different angle, lighting, or background.
- A **bad model** only recognizes the exact training photos and fails on new ones.

The good model has **generalized**; the bad model has **overfit**.

3.2 Overfitting & Underfitting

Overfitting vs Underfitting – The Core ML Challenge

	Overfitting	**Underfitting**
What happens	Model is **too complex**; memorizes training data including noise	Model is **too simple**; fails to capture underlying patterns
Training performance	Very good (low error)	Poor
Test performance	Poor (high error)	Poor
Analogy	Student memorizes exact Q&A; fails on new questions	Student skims material; can't answer even basic questions
Visual	Curve passes through every data point	Straight line through complex curved data
Bias	Low bias	High bias
Variance	High variance	Low variance

3.3 Bias-Variance Tradeoff

Bias-Variance Decomposition

The total error of a model can be decomposed as:

$$\boxed{\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}}$$

Where:

- **Bias** $= E[\hat{f}(x)] - f(x)$: Error from wrong assumptions in the model (simplification error).
- **Variance** $= E[(\hat{f}(x) - E[\hat{f}(x)])^2]$: Error from sensitivity to fluctuations in training data.
- **Irreducible Noise**: Inherent randomness in the data that **cannot** be reduced by any model.

Goal: Find the **sweet spot** — a model complex enough to capture patterns, but simple enough to not memorize noise.

Model Complexity	Bias	Variance
Too simple (underfit)	High	Low
Just right (good fit)	Moderate	Moderate
Too complex (overfit)	Low	High

3.4 Training Data vs Test Data

Training & Test Split

- **Training Data:** Used to *build/train* the model (learn patterns).
- **Test Data:** Used to *evaluate* the model's performance on unseen data.
- **Validation Data:** Used to *tune hyperparameters* during training.

Golden Rule: The test data must **NEVER** be used during training. Using test data during training causes **data leakage** → overly optimistic performance estimates.

3.5 Mean Squared Error (MSE)

Mean Squared Error

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_{\text{true},i} - y_{\text{pred},i})^2$$

Where:

- m = number of data points
- $y_{\text{true},i}$ = actual value of the i-th sample
- $y_{\text{pred},i}$ = predicted value of the i-th sample

The goal of ML is to minimize MSE (or another loss function) on test data.

3.6 Goodness of Fit

What is a Good Fit?

A **good fit** model:

- Captures the underlying pattern/trend in the data
- Does NOT memorize individual training points (avoids overfitting)
- Does NOT ignore the pattern (avoids underfitting)
- **Generalizes well** to new, unseen data

4 Data Preprocessing & Feature Engineering

The ML Lifecycle

Data preprocessing is the **most time-consuming and critical step** in any ML project. The pipeline:

Data Collection → Data Profiling → Data Cleansing → Data Transformation → Data Reduction → Data I

Garbage In = Garbage Out. No model can compensate for bad data!

4.1 Step 1: Data Profiling

Data Profiling

Analyzing data to understand its characteristics:

- Identify the **features** (columns) and their **types** (numerical, categorical)
- Understand **distributions, trends, and ranges** of each feature
- Detect **missing values, duplicates, and outliers**
- Examine **correlations** between features

4.2 Step 2: Data Cleansing

Data Cleansing

Fixing problems in the data:

- **Missing Values:** Replace with mean, median, or mode; or remove the row/column
- **Duplicates:** Remove duplicate records
- **Outliers:** Identify and handle extreme values
- **Inconsistencies:** Fix formatting errors (e.g., "Male" vs "male" vs "M")

Handling Missing Values – Methods

Method	When to Use
Delete rows	When very few rows have missing values
Delete columns	When a column has too many missing values ($> 50\%$)
Mean imputation	For numerical features with roughly normal distribution
Median imputation	For numerical features with skewed distribution
Mode imputation	For categorical features

4.3 Step 3: Data Transformation

Data Transformation Techniques

Converting data into suitable format for modeling:

4.3.1 Feature Scaling

Standardization vs Normalization

Standardization (Z-score):

$$z = \frac{x - \mu}{\sigma}$$

Transforms data to have mean $= 0$ and standard deviation $= 1$.
Normalization (Min-Max Scaling):

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Scales data to range $[0, 1]$.
Why Scale? Features with large ranges (e.g., salary: 30,000–100,000) can dominate features with small ranges (e.g., age: 20–60). Scaling ensures **all features contribute equally**.

4.3.2 Feature Encoding

Encoding Categorical Variables

ML models need **numerical inputs**. Categorical data must be converted:
1. Label Encoding: Assign each category a number.

Category	Label
Red	0
Green	1
Blue	2

Problem: Implies ordering (Blue > Green > Red) which may not exist.
2. One-Hot Encoding: Create binary columns for each category.

Original	Is_Red	Is_Green	Is_Blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1

Preferred for nominal (unordered) categories.

4.4 Step 4: Data Reduction

Data Reduction

Goal: Reduce the number of features while preserving important information.

- Remove **redundant features** (highly correlated with each other)
- Remove **irrelevant features** (don't help prediction)
- Use **dimensionality reduction** techniques (PCA, etc.)

4.5 Step 5: Data Enrichment

Data Enrichment

Creating new, more useful features from existing ones:

- Combine features: e.g., convert (latitude, longitude) → distance from city center
- Create interaction features: e.g., BMI from height and weight
- Extract temporal features: e.g., day of week, hour from timestamp

4.6 Data Leakage

CRITICAL: Data Leakage

Data leakage occurs when **information from the test set is used during training** (directly or indirectly).
Example of leakage:

- Computing mean/std on the *entire* dataset (including test) before splitting
- Feature engineering using information from test data

Prevention:

- **Always split first**, then preprocess
- Use **pipelines** to ensure transformations only use training data
- Compute scaling parameters *only* from training data, apply to test data

5 Cross-Validation & Regularization

5.1 Cross-Validation

Why Cross-Validation?

A single train/test split may give misleading results (you might get lucky or unlucky with the split). **Cross-validation** provides a more **robust and reliable** estimate of model performance.

5.1.1 K-Fold Cross-Validation

K-Fold Cross-Validation

Procedure:

1. Divide data into k **equal-sized folds** (e.g., $k = 5$)
2. For each fold i from 1 to k:
   - Use fold i as **test set**
   - Use remaining $k - 1$ folds as **training set**
   - Train model and record test performance
3. **Final performance** = average of all k test performances

Advantage: Every data point gets to be in the test set exactly once.
Common choice: $k = 5$ or $k = 10$.

5-Fold Cross-Validation Example

With 100 data points and $k = 5$:

Iteration	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	MSE
1	**Test**	Train	Train	Train	Train	0.15
2	Train	**Test**	Train	Train	Train	0.18
3	Train	Train	**Test**	Train	Train	0.12
4	Train	Train	Train	**Test**	Train	0.16
5	Train	Train	Train	Train	**Test**	0.14
					Average MSE:	**0.15**

5.1.2 Leave-One-Out Cross-Validation (LOOCV)

LOOCV

Special case of K-fold where $k = n$ (number of data points).

- Each iteration uses **one data point as test**, rest as training
- Most thorough but **computationally expensive** (n iterations!)
- Used when dataset is very small

5.1.3 Nested Cross-Validation

Nested Cross-Validation

Used when you need to **tune hyperparameters AND evaluate model performance**.
Structure:

- **Outer loop:** K-fold CV to estimate generalization performance
- **Inner loop:** K-fold CV on training data to select best hyperparameters

This prevents "optimistic bias" from using the same data for both tuning and evaluation.

5.2 Regularization

What is Regularization?

Regularization is a technique to **prevent overfitting** by adding a **penalty term** to the loss function that discourages overly complex models (large weights).

$$\text{Total Loss} = \text{Original Loss (e.g., MSE)} + \lambda \times \text{Penalty on weights}$$

Where λ is the **regularization strength** (hyperparameter).

L1 vs L2 Regularization

L1 Regularization (Lasso):

$$\text{Loss} = \text{MSE} + \lambda \sum_{j=1}^{p} |w_j|$$

- Penalty = sum of **absolute values** of weights
- Tends to make some weights **exactly zero** → automatic **feature selection**
- Produces **sparse** models

L2 Regularization (Ridge):

$$\text{Loss} = \text{MSE} + \lambda \sum_{j=1}^{p} w_j^2$$

- Penalty = sum of **squared values** of weights
- Shrinks all weights towards zero but **doesn't make them exactly zero**
- Handles **correlated features** better

	L1 (Lasso)	**L2 (Ridge)**
Penalty type	$\lvert w_j \rvert$ (absolute)	w_j^2 (squared)
Effect on weights	Some become exactly 0	All shrink toward 0
Feature selection	Yes (automatic)	No
When to use	Many irrelevant features	Many correlated features

5.3 Other Techniques to Prevent Overfitting

Prevention Methods

1. **Regularization** (L1/L2) – penalize complexity
2. **Cross-validation** – robust performance estimation
3. **Early stopping** – stop training when validation error starts increasing
4. **Feature selection** – remove irrelevant features
5. **More training data** – harder to memorize with more data
6. **Simpler model** – reduce model complexity (fewer parameters)
7. **Occam's Razor:** Among models with similar performance, prefer the **simplest** one.

6 Feature Selection & Dimensionality Reduction

6.1 Why Feature Selection?

Feature Selection

Feature selection is the process of picking a **subset of significant features** for use in better model construction.
Why?

- Not all features are useful — some add **noise/randomness**
- Reduces **overfitting** risk
- Improves model **accuracy and speed**
- Makes model more **interpretable**

6.2 Feature Selection Methods

Method	How It Works	Pros/Cons
Filter Methods	Select features using **statistical tests** (correlation, chi-square, mutual information) *before* training any model	Fast, model-independent. But may miss feature interactions
Wrapper Methods	Train model with different **subsets of features**, evaluate performance, pick best subset. Example: Recursive Feature Elimination (RFE)	More accurate. But computationally expensive
Embedded Methods	Feature selection happens **during model training**. Example: L1 regularization (Lasso) automatically zeroes out irrelevant features	Best of both worlds. Model-specific

Correlation-Based Filter Method

If features A and B are **highly correlated** ($r > 0.9$), they carry almost the same information. You can **remove one** without losing much predictive power.
This reduces dimensionality and prevents multicollinearity issues.

6.3 Dimensionality Reduction

Dimensionality Reduction

Dimensionality reduction transforms a **high-dimensional** dataset into a **lower-dimensional** representation while preserving the most important information.
Key Insight: Not every dimension is equally important. Some dimensions explain most of the variance in the data.
Difference from Feature Selection:

- Feature Selection: Pick a subset of *original* features
- Dimensionality Reduction: Create *new* features (combinations of originals)

6.3.1 Principal Component Analysis (PCA)

PCA – Key Idea

PCA finds new axes (principal components) that:

1. Are **orthogonal** (perpendicular) to each other
2. Maximize the **variance** captured in each successive component
3. The first principal component captures the **most variance**, the second captures the next most, etc.

Mathematically: Uses **Singular Value Decomposition (SVD):**

$$X = U\Sigma V^T$$

Where:

- X is the data matrix
- U contains eigenvectors (principal components)
- Σ contains eigenvalues (importance of each component)
- Keep only the top k components to reduce from d dimensions to k dimensions

6.4 CRISP-DM Framework

CRISP-DM: Cross-Industry Standard Process for Data Mining

A widely used **6-phase framework** for ML/data mining projects:

#	Phase	Description
1	**Business Understanding**	Define the problem and project objectives
2	**Data Understanding**	Collect data, explore quality and characteristics
3	**Data Preparation**	Clean, transform, engineer features (most time spent here!)
4	**Modeling**	Train and build ML models
5	**Evaluation**	Assess model performance; is it good enough?
6	**Deployment**	Deploy model into production environment

Note: The process is **iterative** — you often go back to earlier phases based on findings.

7 Data Mining & Association Rules

7.1 Data Science vs Data Analytics vs Data Mining

Field	Definition
Data Science	Interdisciplinary field using statistics, scientific computing, and scientific methods to extract knowledge from data. Covers the **complete end-to-end process**.
Data Analytics	Process of **analyzing data** to extract insights and make informed decisions. Uses patterns found in data to drive business outcomes.
Data Mining	Process of **discovering patterns and relationships** in large datasets using algorithms and statistical methods.

7.2 Association Rules

Definition: Association Rule

An **association rule** is a pattern of the form:

$$X \to Y$$

Where X and Y are **sets of items** (itemsets), and $X \cap Y = \emptyset$.
Meaning: "If a transaction/customer has items in X, they are likely to also have items in Y."
Classic Example: "If a customer buys milk, they are likely to also buy bread" $\to$ {milk} $\to$ {bread}

7.3 Measures for Association Rules

Support, Confidence, and Lift

Given a dataset D with $|D|$ total transactions and a rule $X \to Y$:
1. Support:

$$\text{Support}(X \to Y) = \frac{|\{t \in D \mid X \cup Y \subseteq t\}|}{|D|}$$

What fraction of ALL transactions contain both X and Y?
2. Confidence:

$$\text{Confidence}(X \to Y) = \frac{|\{t \in D \mid X \cup Y \subseteq t\}|}{|\{t \in D \mid X \subseteq t\}|} = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Of those who bought X, what fraction also bought Y?
3. Lift:

$$\text{Lift}(X \to Y) = \frac{\text{Confidence}(X \to Y)}{\text{Support}(Y)}$$

How much more likely is Y when X is present vs. Y's baseline probability?
Interpreting Lift:

- Lift > 1: X and Y appear together **more often** than expected (positive association)

- Lift $= 1$: X and Y are **independent**
- Lift < 1: X and Y appear together **less often** than expected (negative association)

Calculating Support, Confidence, Lift

Given 1000 transactions:

- 200 contain Milk
- 300 contain Bread
- 150 contain both Milk and Bread

For the rule: Milk $\rightarrow$ Bread:

$$\text{Support} = \frac{150}{1000} = 0.15 = 15\%$$
$$\text{Confidence} = \frac{150}{200} = 0.75 = 75\%$$
$$\text{Lift} = \frac{0.75}{300/1000} = \frac{0.75}{0.30} = 2.5$$

Interpretation: 15% of all transactions have both. Of milk buyers, 75% also buy bread. Buying milk makes bread purchase 2.5× more likely than baseline.

7.4 Frequent Itemsets & The Apriori Algorithm

Frequent Itemset

A **frequent itemset** is a set of items that appears in the dataset with frequency (support) above a **minimum support threshold**.
Apriori Property (Key Principle):

> If an itemset is **NOT** frequent, then **all its supersets** are also NOT frequent.
> Equivalently: All subsets of a frequent itemset must also be frequent.

This property allows **pruning** of the search space, making the algorithm efficient.

Apriori Algorithm – Steps

Goal: Find all frequent itemsets, then generate association rules.
Step 1: Find all frequent 1-itemsets (single items above min support).
Step 2: Generate candidate 2-itemsets from frequent 1-itemsets.
Step 3: Prune candidates that have an infrequent subset (Apriori property).
Step 4: Count support of remaining candidates, keep frequent ones.
Step 5: Repeat for 3-itemsets, 4-itemsets, etc., until no more frequent itemsets found.
Step 6: Generate rules from frequent itemsets that meet minimum confidence.

Apriori Example

Given transactions and min_support = 2:

TID	Items
T1	{A, B, C}
T2	{A, C}
T3	{B, C, D}
T4	{A, B, C, D}

Frequent 1-itemsets: A:3, B:3, C:4, D:2 (all ≥ 2)
Candidate 2-itemsets: {A,B}:2, {A,C}:3, {A,D}:1, {B,C}:3, {B,D}:2, {C,D}:2
Prune: {A,D} has support $1 < 2$, so remove it.
Frequent 2-itemsets: {A,B}, {A,C}, {B,C}, {B,D}, {C,D}
Continue to 3-itemsets... and so on.

8 Classification & Naive Bayes Classifier

8.1 What is Classification?

Definition: Classification

Classification is a **supervised learning** task where the goal is to predict a **categorical label** (class) for a new data point based on its features.
A Classifier is a function:

$$h : \mathcal{X} \to \mathcal{Y}$$

that maps input features $\mathcal{X}$ to class labels $\mathcal{Y}$.
Types:

- **Binary classification:** 2 classes (e.g., spam/not spam, sick/healthy)
- **Multi-class classification:** > 2 classes (e.g., cat/dog/bird/fish)

8.2 Evaluating a Classifier

Confusion Matrix

For binary classification (Positive vs Negative):

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Evaluation Metrics

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision (of those predicted positive, how many are actually positive?):

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall / Sensitivity (of all actual positives, how many did we catch?):

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score (harmonic mean of precision and recall):

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why Accuracy Alone is Not Enough

In **imbalanced datasets** (e.g., 99% healthy, 1% sick), a model that always predicts "healthy" gets 99% accuracy but **misses all sick patients!**
⇒ Use **Precision, Recall, and F1-Score** in addition to accuracy.

8.3 Bayes Classifier

Bayes' Theorem

$$\boxed{P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}}$$

Where:

- $P(C|X)$ = **Posterior probability** — probability of class C given features X
- $P(X|C)$ = **Likelihood** — probability of observing features X given class C
- $P(C)$ = **Prior probability** — how common is class C overall?
- $P(X)$ = **Evidence** — probability of observing features X (same for all classes)

Bayes Optimal Classifier (MAP Decision Rule)

Maximum A Posteriori (MAP): Assign the class with the **highest posterior probability**:

$$\hat{y} = \arg\max_C P(C|X) = \arg\max_C P(X|C) \cdot P(C)$$

(We can drop $P(X)$ since it's the same for all classes.)
Challenge: Estimating $P(X|C)$ for high-dimensional X requires **enormous amounts of data** (curse of dimensionality).

8.4 Naive Bayes Classifier

Naive Bayes – The "Naive" Assumption

Key Assumption: All features are **conditionally independent** given the class label.

$$P(x_1, x_2, \ldots, x_d|C) = \prod_{i=1}^{d} P(x_i|C)$$

Why "Naive"? This assumption is almost never true in practice. Features are usually correlated. But despite this "naive" assumption, Naive Bayes works surprisingly well!
Classification Rule:

$$\hat{y} = \arg\max_C \left[P(C) \cdot \prod_{i=1}^{d} P(x_i|C) \right]$$

Naive Bayes Example: Spam Detection

Features: words in an email. Classes: Spam, Not Spam.
Suppose we have:

- $P(\text{Spam}) = 0.4, \quad P(\text{Not Spam}) = 0.6$
- Email contains words: "free", "offer"
- $P(\text{"free"}|\text{Spam}) = 0.8, \quad P(\text{"free"}|\text{Not Spam}) = 0.1$
- $P(\text{"offer"}|\text{Spam}) = 0.7, \quad P(\text{"offer"}|\text{Not Spam}) = 0.2$

Compute:

$$P(\text{Spam}|\text{"free", "offer"}) \propto 0.4 \times 0.8 \times 0.7 = 0.224$$
$$P(\text{Not Spam}|\text{"free", "offer"}) \propto 0.6 \times 0.1 \times 0.2 = 0.012$$

Since $0.224 \gg 0.012$, classify as **Spam**.

Advantages & Limitations of Naive Bayes

Advantages:

- Simple and fast
- Works well even with small training data
- Handles high-dimensional data
- Good baseline classifier

Limitations:

- Independence assumption rarely holds
- Cannot capture feature interactions
- Zero probabilities for unseen feature values (solved by **Laplace smoothing**)

9 Quick Reference Summary

9.1 ML Types at a Glance

Type	Data	Goal	Examples
Supervised	Labeled	Learn mapping input → output	Classification, Regression
Unsupervised	Unlabeled	Find hidden patterns	Clustering, PCA
Reinforcement	Reward signal	Maximize cumulative reward	AlphaGo, Robotics

9.2 Key Formulas Cheat Sheet

All Key Formulas

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2 \qquad \text{(Mean Squared Error)}$$

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise} \qquad \text{(Bias-Variance)}$$

$$z = \frac{x - \mu}{\sigma} \qquad \text{(Standardization)}$$

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \qquad \text{(Normalization)}$$

$$\text{Support}(X \to Y) = \frac{|X \cup Y|}{|D|} \qquad \text{(Support)}$$

$$\text{Confidence}(X \to Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)} \qquad \text{(Confidence)}$$

$$\text{Lift}(X \to Y) = \frac{\text{Confidence}(X \to Y)}{\text{Support}(Y)} \qquad \text{(Lift)}$$

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)} \qquad \text{(Bayes' Theorem)}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \qquad \text{(Accuracy)}$$

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{(Precision)}$$

$$\text{Recall} = \frac{TP}{TP + FN} \qquad \text{(Recall)}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \qquad \text{(F1-Score)}$$

$$\text{L1 Loss} = \text{MSE} + \lambda \sum |w_j| \qquad \text{(Lasso)}$$

$$\text{L2 Loss} = \text{MSE} + \lambda \sum w_j^2 \qquad \text{(Ridge)}$$

9.3 Logic Quick Reference

Concept	Summary
Proposition	Statement that is True or False
$\neg p$	NOT p
$p \wedge q$	AND (both true)
$p \vee q$	OR (at least one true)
$p \Rightarrow q$	If p then q (false only when p=T, q=F)
$p \Leftrightarrow q$	p iff q (same truth values)
Valid/Tautology	True for ALL assignments
Satisfiable	True for SOME assignment
Unsatisfiable	False for ALL assignments
CNF	AND of ORs
DNF	OR of ANDs
$p \Rightarrow q \equiv \neg p \vee q$	Implication elimination
Modus Ponens	p, $p \Rightarrow q \vdash q$
Modus Tollens	$\neg q$, $p \Rightarrow q \vdash \neg p$
Sound system	Never proves false things
Complete system	Can prove all true things

9.4 Data Preprocessing Pipeline

Step	Name	What You Do
1	**Data Profiling**	Analyze distributions, find missing values, detect outliers
2	**Data Cleansing**	Handle missing values, remove duplicates, fix inconsistencies
3	**Data Transformation**	Scale features, encode categoricals
4	**Data Reduction**	Remove redundant/irrelevant features
5	**Data Enrichment**	Create new useful features from existing ones
6	**Data Validation**	Verify preprocessing is correct

10 Important Potential Exam Questions

Module 1: AI & ML Basics

1. What is AI? Define it formally.
2. What are the differences between supervised, unsupervised, and reinforcement learning? Give examples.
3. Explain the Turing Test and what capabilities a computer needs to pass it.
4. What is the significance of AlphaGo and AlphaFold in ML history?
5. Discuss ethical concerns in AI with a real-world example (Uber accident).
6. What is a neural network? Explain the perceptron model with the formula.

Module 2: Logic

1. What is a proposition? Give examples and non-examples.
2. Construct truth tables for: $p \Rightarrow q$, $\neg(p \wedge q)$, $(p \vee q) \Rightarrow r$.
3. Prove that $p \Rightarrow q \equiv \neg p \vee q$ using a truth table.
4. What is the difference between validity and satisfiability?
5. Explain soundness and completeness of a logical system.
6. Convert the following to CNF: $(p \Rightarrow q) \wedge (q \Rightarrow r)$.
7. Give an example of a valid argument with a false conclusion.

Module 3: ML Fundamentals

1. Explain overfitting and underfitting with examples.
2. What is the bias-variance tradeoff? Write the decomposition formula.
3. What is generalization? Why is it the goal of ML?
4. Define MSE and explain why we minimize it.
5. What is data leakage and how do you prevent it?

Module 4: Data Preprocessing

1. List and explain the steps in data preprocessing.
2. What is the difference between standardization and normalization? Give formulas.
3. Explain one-hot encoding vs label encoding. When do you use each?
4. How do you handle missing values? List at least 3 methods.
5. What is feature scaling and why is it important?

Module 5: Cross-Validation & Regularization

1. Explain K-fold cross-validation step by step.
2. What is LOOCV? When would you use it?
3. What is nested cross-validation and why is it needed?
4. Compare L1 and L2 regularization (formulas, effects, when to use).
5. What is early stopping? How does it help prevent overfitting?
6. State Occam's Razor in the context of ML.

Module 6: Feature Selection

1. Compare filter, wrapper, and embedded feature selection methods.
2. What is PCA? Explain the intuition behind it.
3. What is the CRISP-DM framework? List its 6 phases.
4. What is dimensionality reduction? How is it different from feature selection?

Module 7: Association Rules & Data Mining

1. Define support, confidence, and lift. Give formulas.
2. Given a transaction dataset, calculate support, confidence, and lift for a rule.
3. Explain the Apriori algorithm step by step.
4. What is the Apriori property and why is it useful?
5. Differentiate between data science, data analytics, and data mining.

Module 8: Classification & Naive Bayes

1. What is classification? Define a classifier mathematically.
2. Draw and explain the confusion matrix. Define TP, FP, TN, FN.
3. Define Accuracy, Precision, Recall, and F1-Score with formulas.
4. Why is accuracy alone not a good metric for imbalanced datasets?
5. State Bayes' Theorem. Explain each component.
6. What is the "naive" assumption in Naive Bayes? Why does it still work?
7. Solve a Naive Bayes classification problem given prior and likelihood values.
8. What is the posterior probability and how is it used for classification?
9. How does dimensionality affect classification? (Curse of dimensionality)

Good luck on your exam! Focus on understanding concepts, not just memorizing.

"Rational thinking says you optimize your exam; but understanding says you optimize your life." – Your Professor